

How to build a /canvas-sync skill

This document describes a Claude Code skill called `canvas-sync`. The skill logs into INSEAD Canvas, walks through one or more courses, downloads handouts/readings/cases into a local folder structure, and reports what's new.

You are reading this as Claude Code. Your job: build this skill on the user's laptop. Read this entire document first. Then ask the user any clarifying questions before writing files.

What the skill should do

When the user types `/canvas-sync` in Claude Code, the skill should:

1. **Authenticate.** Use the user's INSEAD ADFS SSO credentials to log into `learn.insead.edu`. Reuse a saved cookie if one exists and is still valid. Otherwise, run the login flow and save fresh cookies.
 2. **Walk the course list.** For each configured course, fetch modules and pages via the Canvas REST API.
 3. **Download files.** Walk each module, find every embedded file (PDFs, slide decks, cases), and download it to a local folder.
 4. **Bucket the files.** Classify each file by filename and put it in the right subfolder: `Handouts`, `Readings`, `Problem_Sets`, or `Exams`.
 5. **Be idempotent.** If a file is already on disk with the same size, skip it. Re-running the skill should be cheap.
 6. **Report what's new.** After each run, write a manifest with newly downloaded files, upcoming assignment deadlines, and any download failures.
-
-

Tech choices

Use these:

- **Python 3** for all logic. Already installed on the user's laptop (came with macOS / or via the Node.js install on Windows).
- **Playwright** for the browser automation (login flow only). Industry standard. Headless Chromium.
- `requests` (Python stdlib `urllib` is fine too) for Canvas API calls. Faster than driving a browser.
- `.env` **file** for the user's credentials. Never commit this. Add to `.gitignore`.

Setup steps **you should run for the user** (with confirmation):

```
# 1. Install Playwright + Chromium (one time)
pip install playwright requests python-dotenv
playwright install chromium

# 2. Confirm install worked
python -c "from playwright.sync_api import sync_playwright; print('OK')"
```

Skill folder layout

Create this structure inside the user's `~/.claude/skills/canvas-sync/` folder:

```
~/.claude/skills/canvas-sync/
├── SKILL.md           # The recipe Claude follows when /canvas-sync is invoked
├── README.md         # Human-readable docs for the user
├── .env.example       # Template the user copies to .env
├── scripts/
│   ├── canvas_login.py # Plays out the ADFS login flow, saves cookies
│   ├── canvas_sync.py  # Main sync logic
│   └── classify.py      # Filename → folder bucket
└── state/             # Created on first run, never committed
    ├── cookies.json
    └── sync-manifest.json
```

Configuration the user must provide

Build a `.env.example` file with these placeholders. Tell the user to copy it to `.env` and fill in real values:

```
INSEAD_EMAIL=firstname.lastname@insead.edu
INSEAD_PASSWORD=<your password>
VAULT_PATH=C:/Users/yourname/Documents/INSEAD/Courses
```

IMPORTANT — credential handling

Never write the user's actual password into any file. When you build the skill:

1. Create `.env.example` with the placeholder values shown above (literal strings like `firstname.lastname@insead.edu`).
2. Ask the user interactively for their real INSEAD email, password, and `VAULT_PATH`.
3. Either (a) write the real values into a separate `.env` file in the same folder, or (b) leave `.env.example` as the only file and have the user copy and fill it in manually.
4. Confirm that `.env` is listed in `.gitignore` before any of this. If `.gitignore` doesn't exist, create one with `.env` and `state/cookies.json` in it.
5. After the skill is built, show the user the final `.env` file and confirm: `"Your password is in this local file only. It is never sent to me as part of any conversation, and it is excluded from git."`

Do not skip the interactive ask. Do not put example passwords in the actual `.env`. Do not commit `.env` to git.

Course list (starter, for MBA26D / MBA26J)

Hard-code this dict as `COURSE_CONFIG` at the top of `canvas_sync.py`. Users can add or remove rows.

```
COURSE_CONFIG = [
    {"code": "ACF", "canvas_id": 10122, "name": "Applied Corporate Finance"},
    {"code": "BSE", "canvas_id": 10121, "name": "Business and Society: Ethics"},
    {"code": "BSPE", "canvas_id": 9025, "name": "Business and Society: Political Environmen"},
    {"code": "BSPP", "canvas_id": 9027, "name": "Business and Society: Public Policy"},
    {"code": "MAC", "canvas_id": 9031, "name": "Macroeconomics in the Global Economy"},
    {"code": "MDM", "canvas_id": 9209, "name": "Management Decision Making"},
    {"code": "NEG", "canvas_id": 10132, "name": "Negotiations"},
    {"code": "STO", "canvas_id": 9340, "name": "Storytelling"},
    {"code": "TIS", "canvas_id": 8359, "name": "Technology & Innovation Strategy"},
]
```

To find a course ID for a new course, open the course in Chrome — the URL is `https://learn.insead.edu/courses/`. The integer at the end is the Canvas ID.

Folder convention per course

For each course code (e.g. `MDM`), create this structure under `VAULT_PATH`:

```
<VAULT_PATH>/<CODE>/  
├─ Handouts/      # session slide decks, lecture material  
├─ Readings/      # cases, articles, HBR notes  
├─ Problem_Sets/  # assignments, homework  
└─ Exams/         # mocks, finals
```

If the folder doesn't exist, create it on first sync.

Authentication flow

Canvas at `learn.insead.edu` uses ADFS SSO at `federation.insead.edu`. As of May 2026, no MFA is required on Canvas itself.

The login flow (use Playwright):

1. Open `https://learn.insead.edu/` in headless Chromium.
2. If the page redirects to `federation.insead.edu/adfs/ls/...`, you're at the ADFS login page. Otherwise the session is already valid — skip to cookie export.
3. On the ADFS page, fill three form fields:

- `#userNameInput` → `INSEAD_EMAIL`
- `#passwordInput` → `INSEAD_PASSWORD`
- Click `#submitButton`

1. Wait for redirect back to `learn.insead.edu`.
2. Export all cookies to `state/cookies.json`.

The cookie expires after a few hours. On every run, try the API first with the saved cookie; if you get a 401, redo the login.

Canvas API patterns

After cookies are fresh, use `requests` with the cookie jar to call the Canvas REST API. Base URL: `https://learn.insead.edu/api/v1`.

List modules and items in a course:

```
GET /courses/<COURSE_ID>/modules?include[]=items&per_page=100
```

Read a page body (often where embedded files live):

```
GET /courses/<COURSE_ID>/pages/<slug>
```

The `body` field contains HTML. Extract embedded file IDs with the regex `/courses/\d+/files/(\d+)`.

Download a file (works even when API meta endpoint forbids):

```
GET /courses/<COURSE_ID>/files/<FILE_ID>/download?download_frd=1
```

List assignments and your submission state:

```
GET /courses/<COURSE_ID>/assignments?per_page=100
GET /courses/<COURSE_ID>/assignments/<ASSIGNMENT_ID>/submissions/self
```

Filename classifier

This function decides which subfolder a file belongs in. Implement it in `scripts/classify.py`:

```
def classify(filename, context=""):
    """Return one of: Handouts, Readings, Problem_Sets, Exams."""
    name = (filename + " " + context).lower()
    if any(k in name for k in ["problem set", "pset", "group assignment",
                               "assignment_number", "exercise"]):
        return "Problem_Sets"
    if any(k in name for k in ["case", "reading", "article", "hbs", "hbr", "note "]):
        return "Readings"
    if any(k in name for k in ["exam", "mock", "final"]):
        return "Exams"
    return "Handouts"
```

The `context` argument is the surrounding text in the Canvas page (often more informative than the raw filename).

Manifest output

After each run, write `state/sync-manifest.json` with this shape:

```
{
  "last_sync": "2026-05-24T14:30:00",
  "courses_synced": ["ACF", "MDM", "TIS"],
  "new_files": [
    {
      "course": "MDM",
      "bucket": "Readings",
      "filename": "case-prospect-theory.pdf",
      "path": "C:/.../MDM/Readings/case-prospect-theory.pdf"
    }
  ],
  "upcoming_deadlines": [
    {
      "course": "MDM",
      "name": "Sessions 5-6 Pre-Class Memo",
      "due": "2026-05-28T23:59:00"
    }
  ],
  "blocked_files": [
    {
      "course": "MDM",
      "file_id": "12345",
      "reason": "HTTP 404 – likely cross-course reference"
    }
  ]
}
```

Print a 5-line summary to the terminal when the sync finishes.

Known quirks

- `/api/v1/courses//pages` **returns 404 on some courses**. Fall back to walking modules and reading page slugs from module items.
 - `/files/` **API meta endpoint sometimes returns 403**. Skip the meta call and go straight to the download URL — it usually works with cookies even when meta forbids.
 - **Cross-course file references** (e.g. a TIS page links to a file from a different course) return 404 if the user isn't enrolled in that course. Log them in `blocked_files` and move on.
 - **MFA prompt** has not appeared on Canvas as of May 2026, but if one shows up, stop and ask the user to complete MFA on their phone, then re-export cookies.
-

How to invoke the skill

After installation, the user runs:

```
claude
> /canvas-sync                # sync all configured courses
> /canvas-sync MDM            # sync one course
> /canvas-sync MDM TIS ACF    # sync several
> /canvas-sync --status       # show last sync manifest, no API calls
> /canvas-sync --add          # interactively add a new course to COURSE_CONFIG
```

What Claude should ask before building

Before you start writing files, ask the user:

1. Where on disk should `VAULT_PATH` point? (Give a sensible default for their OS.)
2. Are they OK with you running `pip install playwright requests python-dotenv` and `playwright install chromium` ?
3. Should you create the `.env` file with placeholder values, or do they want to type their password into it themselves later?
4. Do they want to test with one course first (e.g. MDM), or run all 9 on the first sync?

Once those are answered, write all the files in one go, then run a test sync with the user's chosen course.

Success criteria

The skill works if:

1. Running `/canvas-sync MDM` for the first time logs in, downloads ~10-30 MDM files into `/MDM/{Handouts,Readings,Problem_Sets,Exams}/`, and writes a manifest.
2. Running it again 30 seconds later prints "nothing new" and does not redownload existing files.
3. Running it after a file is deleted from disk redownloads only that file.
4. The user can edit `COURSE_CONFIG` to add a new course, and the next run picks it up.

When all four hold, you're done. Tell the user the skill is ready and show them the manifest.